Agent Reliability Lab: Controlled Fault Injection and Evidence-Based Failure Analysis for Tool-Using Language Agents

Suryansh Singh
Department of Computer Science and Engineering
Noida Institute of Engineering and Technology
Greater Noida, India
0231cse087@niet.co.in

Simran Kumari
Department of Computer Science and Engineering
Noida Institute of Engineering and Technology
Greater Noida, India
0231cse179@niet.co.in

Tanishka Sahu
Department of Computer Science and Engineering
Noida Institute of Engineering and Technology
Greater Noida, India
0231cse196@niet.co.in

***Abstract*—Tool-using agents built on large language models plan, call external tools, read observations, and retry after errors. Most evaluations still report a single success rate from one attempt per task in a clean environment, which hides why runs fail, how consistently an agent succeeds, and what reliability costs. This paper presents Agent Reliability Lab, an open-source Python laboratory for controlled reliability experiments on any agent that maps a task to a recorded trajectory. The laboratory injects tool timeouts and outages on a seeded schedule that depends only on the task, trial, tool, and call number, so every agent variant meets exactly the same failures and their runs can be compared as matched pairs. It runs each task several times and reports success with confidence intervals, the probability that all attempts succeed, consistency, cost, latency, recovery, and step use. A rule-based classifier explains each failed run as a planning, tool-misuse, context, recovery, or horizon failure with evidence, and leaves unmatched failures unclassified. A configuration search returns the cheapest setting that meets a reliability target. In preliminary experiments with offline reference agents on a twelve-task customer-support suite, the measured effect of injected faults matches the closed-form expectation, the classifier recovers every planted defect with perfect precision, and the search identifies a retry budget that keeps success above ninety-five percent under thirty percent tool timeouts.**

***Index Terms*—agent evaluation, fault injection, failure analysis, reliability, tool use, large language models**

I. Introduction

Large language model (LLM) agents no longer answer a single prompt. Following the reasoning and acting pattern of ReAct [1], they decompose a goal, call tools, read the results, and continue for many steps. Benchmarks such as AgentBench [2], GAIA [3], SWE-bench [4], WebArena [5], and τ-bench [6] have made agent capability measurable. They mostly answer one question: did the agent finish the task?

Deployed agents face a harder question: will the agent finish the task every time, in a world where tools time out, and at what cost? A single success rate cannot answer it. Two agents with the same score may fail for unrelated reasons, one by guessing instead of calling a required tool and another by abandoning a task after one timeout. Success measured once per task also hides inconsistency; on τ-bench the chance that a strong model solves the same retail task in all of eight attempts falls below 25% even though its single-attempt success exceeds 60% [6]. Kapoor *et al.* further showed that ignoring cost leads to needlessly complex agents and mistaken conclusions about where accuracy gains come from [9].

This paper describes Agent Reliability Lab (relai), a laboratory in which these questions become controlled experiments. Its contributions are:

1) A small agent interface, a name plus a run method that returns a trajectory, which admits scripted policies, hand-written agents, and LLM agents behind any OpenAI-compatible endpoint without changing the harness.
2) Seeded, agent-independent fault injection: each tool call fails or succeeds as a deterministic function of the seed, task, trial, tool, and call number. All variants meet the same failure schedule, which makes paired significance tests valid.
3) Repeated-trial reliability metrics with uncertainty: pass@k, pass^k, consistency, Wilson and task-clustered bootstrap intervals, and an exact McNemar test between variants.
4) An evidence-producing failure taxonomy with five modes and an explicit unclassified outcome, validated against reference agents whose defects are planted on purpose.
5) Cost- and latency-aware configuration search that returns the Pareto front and the cheapest setting that meets a reliability target.

Section II reviews related work, Section III states the

problem, Section IV describes the methodology, Section V reports preliminary results, and Section VI concludes.

II. Related Work

A. Agent Benchmarks and Reliability Metrics

AgentBench [2], GAIA [3], SWE-bench [4], and WebArena [5] contribute tasks and environments, and report task success. τ-bench [6] introduced pass^k, the chance that all of k independent trials succeed, which complements the pass@k estimator of Chen *et al.* [8]; its successor τ^2-bench adds dual-control environments [7]. The Holistic Agent Leaderboard (HAL) standardizes cost-aware evaluation across benchmarks and uses LLM-aided log inspection [10], and Rabanser *et al.* decompose reliability into consistency, robustness, predictability, and safety with twelve metrics [11]. Agent Reliability Lab adopts pass^k and cost awareness from this line of work and adds paired statistical comparison and per-run diagnosis.

B. Fault Injection

Chaos engineering deliberately injects failures into production systems to build confidence in their resilience [12]. For agents, ToolEmu emulates tools with an LLM to expose risky behavior [13], ToolMisuseBench offers replayable tool faults under call and retry budgets [14], ToolBench-X injects recoverable tool-environment hazards [15], and AgentChaos injects faults into LLM API responses through a proxy [16]. These works are benchmarks or injection layers. Agent Reliability Lab treats the fault schedule as a controlled experimental factor that is identical across variants and checks the injector against its analytic expectation.

C. Failure Taxonomies and Attribution

MAST derives fourteen failure modes for multi-agent systems from annotated traces [17], TRAIL annotates errors in long agent traces [18], and Who&When attributes failures to a responsible agent and step [19]. These efforts rely on human annotation or LLM judges, and the best automated attribution identifies the failing step in only 14.2% of cases [19]. Agent Reliability Lab uses deterministic rules over structured trajectories, reports the evidence behind each label, and refuses to label what no rule explains.

D. Evaluation Frameworks

General frameworks such as HELM [20], Inspect [21], and tracing platforms such as LangSmith and Langfuse record and score runs at scale. They are complementary: Agent Reliability Lab is a small library for controlled experiments that can run on a laptop with no model access.

III. Problem Statement

Let $\mathcal{T}$ be a suite of tasks, each with a prompt, an expected answer, a set of permitted tools, and an optional step budget B_t. Let $\mathcal{V}$ be a set of agent variants, such as different models, prompts, or retry policies. An environment exposes tools whose calls may fail according to a fault process F with seed s. Running variant v on task t in trial i produces a trajectory $\pi_{v,t,i}$: an ordered sequence of plan, tool call, observation, error, recovery, and final steps, each with optional latency, token, and cost annotations.

The problem is to design a laboratory that, for every $v \in \mathcal{V}$:

1) estimates success, pass^k, consistency, cost, latency, recovery, and step use with honest uncertainty;
2) explains each failed run by a root cause with evidence, without forcing unexplained failures into a category;
3) decides whether two variants differ significantly under identical faults; and
4) selects the configuration that minimizes cost subject to a reliability target.

Any callable that maps a task to a trajectory must be admissible, and scoring and analysis must read trajectories only, so that every number can be reproduced from stored runs.

IV. Proposed Methodology

Fig. 1 shows the architecture. Inputs are a task suite and an agent. The harness runs every variant on every task for k trials, routing tool calls through a fault-injecting environment. Scorers, metric derivation, and the failure classifier read the finished trajectories. Statistics, optimization, storage, and reporting operate on the resulting reports.

A. Trajectory Model and Agent Interface

A trajectory is a list of typed steps. Tool-call steps carry the tool name, arguments, and an ok flag; an error step that follows a failed call marks an environment fault. An agent needs only a name and a run method. Adapters wrap plain functions, replay scripted trajectories, or drive an LLM through an OpenAI-compatible chat API, where a recorder converts every model turn and tool call into steps with token, cost, and latency annotations. Exceptions raised by an agent become failed runs, so one crash never aborts an experiment.

B. Seeded, Agent-Independent Fault Injection

For the n-th call to tool τ in trial i of task t under seed s, the environment draws

$$u = h(s, t, i, \tau, n) \in [0, 1), \tag{1}$$

where h is a deterministic pseudo-random function of its arguments. The call times out when n is within a fail-first window or $u < p_{\mathrm{to}}$, returns an outage when $p_{\mathrm{to}} \le u < p_{\mathrm{to}} + p_{\mathrm{err}}$, and runs the tool otherwise. Because h does not depend on the agent, two variants that make the same calls meet the same faults. This is the method of common random numbers from simulation practice: runs of different variants form matched pairs, which reduces variance and licenses paired tests. Calls to unknown tools or with invalid arguments fail without an error step, so they are recorded as misuse by the agent rather than as faults of the environment.

For an agent that retries each failed call up to r times, a task that needs L calls succeeds with probability

$$P(\text{success}) = \left(1 - p^{\,r+1}\right)^{L}, \tag{2}$$

when faults are independent with rate p. Section V uses (2) as a check that the injector behaves as specified.

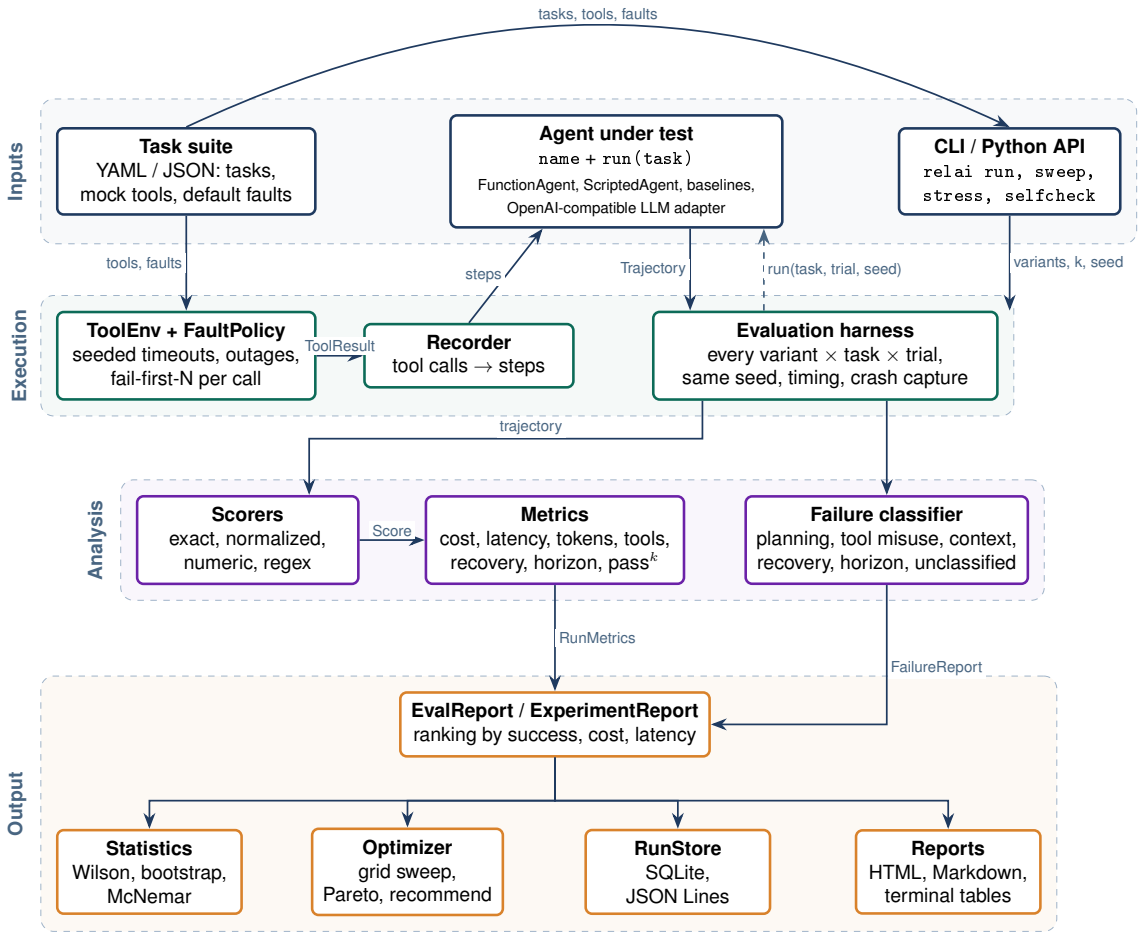


Fig. 1. Architecture of Agent Reliability Lab. Only the execution layer calls agents; every later layer reads recorded trajectories.

C. Repeated Trials and Reliability Metrics

Each task runs for n trials. With c successes on a task,

$$\text{pass@}k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}, \qquad \text{pass}^k = \frac{\binom{c}{k}}{\binom{n}{k}}, \tag{3}$$

averaged over tasks [6], [8]. pass@k measures capability, the chance that at least one of k attempts works, while pass^k measures dependability, the chance that all of them work. A task is consistent when its trials all pass or all fail. Per run, the laboratory also derives cost, latency, tokens, tool calls, repeated calls that ignore an observation, step count, and the recovery rate, the share of error steps after which the run later restored a working path.

D. Failure Taxonomy

Five detectors read each scored trajectory:

- *Planning*: an unsuccessful run with a final answer and no tool failure that skipped required tools, lacked a plan where one was required, or answered wrongly without any tool.
- *Tool misuse*: a call to a tool outside the permitted set, or a failed call that is not followed by an environment error.
- *Context*: the same tool with the same arguments called again immediately after its observation.
- *Recovery*: an error after which no later call of the failed tool succeeds and the run does not end with a correct answer. Failed retries do not close the episode, so a retry chain that ends in success counts as recovered.
- *Horizon*: no final answer, or more steps than the budget B_t.

Each detector returns the index of its first evidence and a plain-language reason. Several modes may fire on one run; the root cause is the mode with the earliest evidence, because the first breakdown usually explains later symptoms. A failed run that no detector matches is reported as unclassified. Every rule is pinned by trajectory fixtures that show both when it fires and when it must stay silent.

E. Statistical Comparison

With one trial per task, success intervals use the Wilson score interval [22]. With repeated trials, runs of a task are

correlated, so a seeded bootstrap [23] resamples whole tasks. Two variants are compared by pairing their runs on (task, trial) and applying the exact McNemar test [24] to the discordant pairs, a standard choice for comparing two classifiers on the same examples [25].

F. Configuration Search

A sweep evaluates one variant per point of a parameter grid under identical conditions. Variant a dominates b when it is no worse on success, cost, and latency and better on at least one. The Pareto front contains the non-dominated variants, following the accuracy and cost frontier of [9], and the recommendation is the cheapest, then fastest, variant whose success meets a target.

G. Implementation

The laboratory is a Python 3.11+ package of about 3,300 lines with two runtime dependencies, Pydantic and PyYAML. Task suites are YAML files that declare tasks, mock tools, default faults, and reference tool plans. Results persist in SQLite and JSON Lines and render as standalone HTML. A command-line interface exposes run, sweep, stress, self-check, and report commands. Eighty-one automated tests cover every module and run in under one second.

V. Preliminary Results

All results in this section use offline reference agents, not LLMs, so that the ground truth of every run is known; they validate the instrument rather than rank models. The suite is a customer-support desk of twelve tasks needing zero to five tool calls. Unless noted, runs use five trials per task, seed 0, and a tool timeout rate of 0.3. The naive agent never retries; retry agents retry each faulted call up to one or three times. Every number can be regenerated with one script in the repository.

A. The Fault Injector Matches Its Specification

Fig. 2 compares measured success over 200 trials per task with (2). With no retry, measured success on one-, three-, and five-call tasks is 0.69, 0.33, and 0.16 against expected values of 0.70, 0.34, and 0.17; with one retry the corresponding values are 0.90, 0.76, and 0.60 against 0.91, 0.75, and 0.62. The injector therefore produces faults at the configured rate and independently across calls, and success decays geometrically with task length.

B. Reliability Under Faults

Table I reports the three agents. Single-attempt success hides most of the difference in dependability: one retry raises success from 0.58 to 0.77 but pass^5 only from 0.25 to 0.33, while three retries reach 1.00 on both at about twice the cost and latency of the naive agent. The paired test separates three retries from one retry with 14 discordant runs, all favoring three retries ($p = 1.2 \times 10^{-4}$), and from no retry with 25 ($p = 6.0 \times 10^{-8}$). The diagnosis also shows the price of retrying: twelve runs by the three-retry agent answered correctly but exceeded their step budget and were flagged as horizon failures.

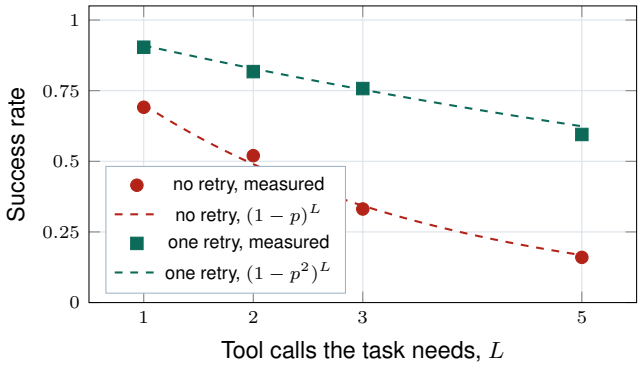


Fig. 2. Measured success against the closed-form expectation (2) at timeout rate 0.3, 200 trials per task.

TABLE I
Agents Under 30% Tool Timeouts (12 Tasks, 5 Trials)

Agent	Success [95% CI]	pass^5	Cost ($)	Lat. (ms)	Root causes
naive	0.58 [0.38, 0.77]	0.25	0.0016	543	25 recovery
retry, 1	0.77 [0.67, 0.88]	0.33	0.0025	947	14 recovery
retry, 3	1.00 [1.00, 1.00]	1.00	0.0030	1116	12 horizon

Fig. 3 sweeps the timeout rate from 0 to 0.5. The gap between success and pass^5 widens with the fault rate for every agent, which is the inconsistency a single attempt per task cannot reveal.

C. The Classifier Recovers Planted Defects

Each reference agent carries one planted defect: no retry, answering from memory, repeating calls, or calling a forbidden tool, plus a retrying control agent. Table II counts the root cause diagnosed in each of 60 runs per agent. Every diagnosed run of a planted agent is attributed to its planted mode, giving a precision of 1.00 for all four modes. Runs counted as none are runs where the defect never surfaced, for example a naive agent that met no timeout. The control agent shows one genuine recovery failure, three consecutive timeouts on one call, and eleven step-budget overruns.

D. Choosing a Retry Budget

A sweep over zero to four retries at a 0.3 timeout rate places every setting on the Pareto front, because each extra retry buys success with cost and latency. Success rises 0.58, 0.77, 0.98, 1.00, and 1.00, with pass^5 of 0.25, 0.33, 0.92, 1.00, and 1.00. For a target of 0.95 the recommendation is two retries, the cheapest setting that meets it.

E. Limitations

The reference agents follow fixed tool plans, so these experiments test the instrument, not the reasoning of LLMs. Tools are mock lookups, faults are limited to timeouts and outages, and the rule-based classifier cannot judge the content of free-text reasoning. The next phase runs hosted and local LLMs through the included adapter and adds suites adapted from public benchmarks.

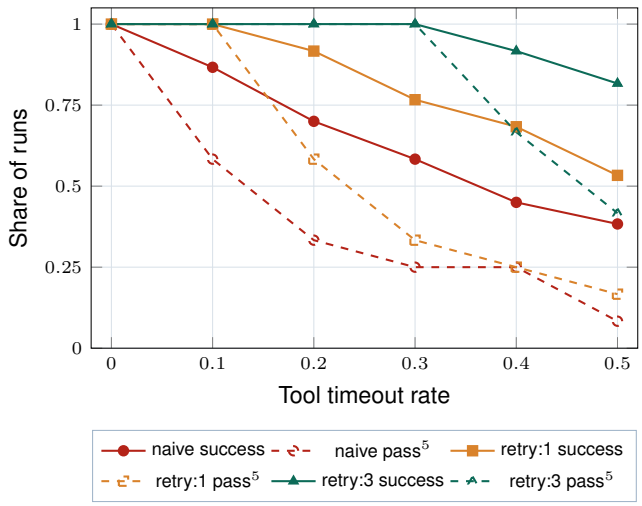


Fig. 3. Success (solid) and pass^5 (dashed) as the tool timeout rate rises.

TABLE II
DIAGNOSED ROOT CAUSES PER PLANTED DEFECT (60 RUNS EACH)

Agent	Planted	Plan.	Misuse	Ctx.	Recov.	Horiz.	None
naive	recovery	0	0	0	25	0	35
guessing	planning	55	0	0	0	0	5
looping	context	0	0	55	0	0	5
careless	misuse	0	60	0	0	0	0
retry	control	0	0	0	1	11	48

VI. CONCLUSION

Agent Reliability Lab turns the question of whether an agent works into controlled experiments on how consistently it works, why it fails, and what reliability costs. Agent-independent fault schedules make variant comparisons paired, repeated trials expose inconsistency that single attempts hide, and evidence-based diagnosis explains each failure without inventing labels. Preliminary experiments confirm that the fault injector matches its analytic expectation and that the classifier recovers planted defects. Future work will evaluate live LLM agents, add fault kinds such as rate limits and corrupted outputs, import traces from tracing platforms, and study whether the taxonomy transfers to multi-agent systems.

ACKNOWLEDGMENT

The authors thank Prof. Tripti Sharma, Head of the Department of Computer Science and Engineering, Noida Institute of Engineering and Technology, Greater Noida, for her guidance and support, and Mr. Pradeep Kumar, Assistant Professor, for his supervision and technical feedback throughout this project.

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
[2] X. Liu *et al.*, "AgentBench: Evaluating LLMs as agents," in *Proc. ICLR*, 2024.
[3] G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom, "GAIA: A benchmark for general AI assistants," in *Proc. ICLR*, 2024.
[4] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. ICLR*, 2024.
[5] S. Zhou *et al.*, "WebArena: A realistic web environment for building autonomous agents," in *Proc. ICLR*, 2024.
[6] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," arXiv:2406.12045, 2024.
[7] V. Barres, H. Dong, S. Ray, X. Si, and K. Narasimhan, "τ^2-bench: Evaluating conversational agents in a dual-control environment," arXiv:2506.07982, 2025.
[8] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv:2107.03374, 2021.
[9] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, and A. Narayanan, "AI agents that matter," arXiv:2407.01502, 2024.
[10] S. Kapoor *et al.*, "Holistic agent leaderboard: The missing infrastructure for AI agent evaluation," in *Proc. ICLR*, 2026.
[11] S. Rabanser, S. Kapoor, P. Kirgis, K. Liu, S. Utpala, and A. Narayanan, "Towards a science of AI agent reliability," in *Proc. 43rd Int. Conf. Mach. Learn. (ICML)*, PMLR, vol. 306, 2026.
[12] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, "Chaos engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.
[13] Y. Ruan *et al.*, "Identifying the risks of LM agents with an LM-emulated sandbox," in *Proc. ICLR*, 2024.
[14] "ToolMisuseBench: An offline deterministic benchmark for tool misuse and recovery in agentic systems," arXiv:2604.01508, 2026.
[15] "Beyond function calling: Benchmarking tool-using agents under tool-environment unreliability," arXiv:2606.25819, 2026.
[16] "AgentChaos: Chaos engineering for agent systems via programmatic fault injection," arXiv:2608.06790, 2026.
[17] M. Cemri *et al.*, "Why do multi-agent LLM systems fail?" arXiv:2503.13657, 2025.
[18] D. Deshpande, V. Gangal, H. Mehta, J. Krishnan, A. Kannappan, and R. Qian, "TRAIL: Trace reasoning and agentic issue localization," arXiv:2505.08638, 2025.
[19] S. Zhang *et al.*, "Which agent causes task failures and when? On automated failure attribution of LLM multi-agent systems," in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, PMLR, vol. 267, 2025.
[20] P. Liang *et al.*, "Holistic evaluation of language models," *Trans. Mach. Learn. Res.*, 2023.
[21] UK AI Security Institute, "Inspect: A framework for large language model evaluations," 2024. [Online]. Available: https://inspect.aisi.org.uk
[22] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.
[23] B. Efron, "Bootstrap methods: Another look at the jackknife," *Ann. Statist.*, vol. 7, no. 1, pp. 1–26, 1979.
[24] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
[25] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Comput.*, vol. 10, no. 7, pp. 1895–1923, 1998.